PFE

Primenjena Fizika i Elektronika

Zimski kamp za starije polaznike

Vežba

Floorplanning i mreža napajanja

N. Petrović

Mart 2026

Vežve su bazirane na: https://pulp-platform.org/efclwinter2026/

Uvod

Cilj ove vežbe je da vidimo sledeće:

- kako se određuje veličina čipa,
- kako se postavljaju U/I padovi,
- kako se postavljaju makroćelije,
- kako se projektuje mreža za distribuciju napajanja.

Ovaj proces naziva se *floorplanning*.

U standardnom dizajnu, primarni cilj je odrediti raspored (*floorplan*) koji obezbeđuje minimalnu površinu, a da pri tom budu zadovoljeni svi zahtevi vezani za performanse i pouzdanost. Razlog za to je čisto ekonomske prirode, jer je cena proizvodnje čipa u velikoj meri uslovljena njegovom površinom. Ponekad postoje i dodatna geometrijska ograničenja koja nameće proizvođač. *Back-end* projektovanje ne primenjuje se isključivo na kompletne čipove. Makroćelije namenjene integraciji u veći *system-on-chip* dizajn takođe mogu biti projektovane na ovaj način. U takvim slučajevima ograničenja mogu biti još strožija; na primer, pojedini metalni slojevi mogu biti rezervisani za sistemski nivo.

Pre nego što pređemo na vežbe, podsetimo se nekih osnovnih pojmova.

Površina čipa (*die area*) predstavlja ukupnu površinu silicijuma koja je na raspolaganju.

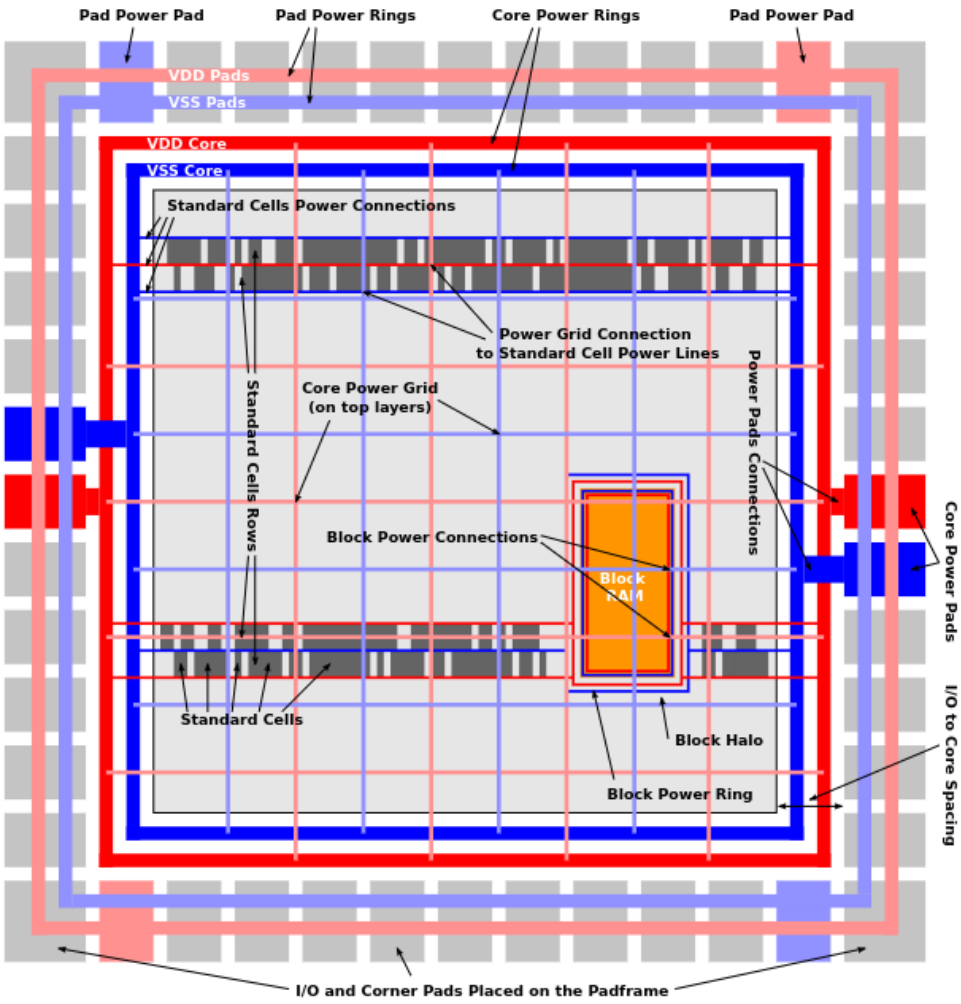
Svi pad-ovi (I/O, pad-ovi za napajanje i ugaoni pad-ovi) smeštaju se u *padframe*, koji čini dodatni hijerarhijski nivo u kome se instanciraju i sam dizajn i I/O ćelije.

Preostala površina namenjena je jezgru čipa (*core area*).

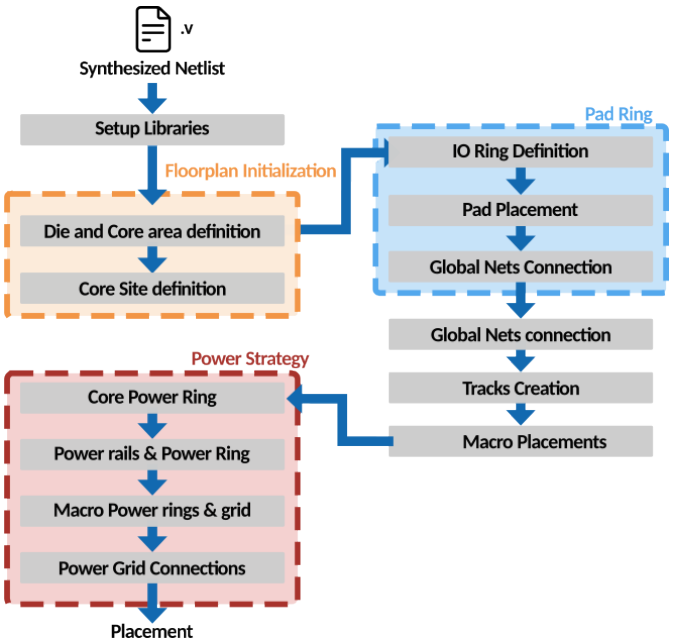
Oblast jezgra okružena je prstenom za napajanje (*core power ring*) čija je uloga ravnomerna distribucija napajanja po celom čipu. U najjednostavnijem obliku, taj prsten se sastoji od dve široke metalne linije — jedne za VDD i druge za VSS. Između I/O oblasti i jezgra neophodno je ostaviti dovoljno prostora za smeštanje ovog prstena.

Oblast jezgra ispunjena je redovima standardnih ćelija u koje se postavljaju sve standardne ćelije. Iskorišćenost (*utilization*) označava procenat popunjenosti tih redova. Standardne ćelije su projektovane tako da se, prilikom postavljanja jedne pored druge, njihovi VDD i VSS pinovi mogu međusobno povezati po jednom horizontalnom metalnom linijom. Ove linije za napajanje su relativno uske i protežu se celom širinom jezgra kako bi se spojile sa prstenovima za napajanje. Kod dizajna sa velikim utroškom struje to može predstavljati problem, jer ćelije bliže središtu jezgra nemaju dovoljno kvalitetnu vezu sa napajanjem. Kao rešenje, dva najviša metalna sloja obično se koriste za formiranje mreže za napajanje iznad oblasti jezgra. Vertikalne i horizontalne trake te mreže povezuju se nadole kroz sve metalne slojeve do uskih linija za napajanje standardnih ćelija.

U oblasti jezgra takođe je potrebno predvideti prostor za makroćelije. Većina makroćelija zahteva određen slobodan prostor u svom okruženju — radi ostvarivanja signalnih veza, dodavanja blokovskog prstena za napajanje ili jednostavno sprečavanja postavljanja standardnih ćelija preblizu makrou. Taj slobodan prostor definiše se pomoću parametra *block halo*.



Slika 1: Primer floorplan-a.



Slika 2: Primer floorplan toka projektovanja.

0.1 O stilu

Zadatak za studente:

Delovi teksta sa sivom pozadinom označavaju korake potrebne za završavanje vežbe.

Tokom vežbe unosite komande putem komandne linije[1].

0.2 Početak rada

Zadatak 1:

- Pokrenite docker

```
sh > ./run_docker.sh
```

- Prebacite se u direktorijum:

```
sh > cd exercises/asic/openroad
```

Napomena 1: Ako otvorite novi terminal, ne zaboravite da ponovo pokrenete docker.

Napomena 2: Celu vežbu radimo u `openroad` folderu.

1 OpenROAD konfiguracija

Napomena 3: Korake ove vežbe je potrebno da upisujete i u `OpenROAD` terminal i u praznu `tcl` skriptu koja je pripremljena za vas i nalazi se u `scripts/01_pfe_floorplan.tcl`. Ne zaboravite, nalazimo se u `openroad` folderu.

Ukoliko ste pisali skriptu (a jeste), ukoliko nešto pogrešite lako možete ponovo pokrenuti sve komande pokretanjem `tcl` skripte u OpenROAD terminalu:

```
OpenROAD > source scripts/01_pfe_floorplan.tcl
```

ili direktno iz terminala:

```
sh > openroad -gui scripts/01_pfe_floorplan.tcl
```

Zadatak 2: Setup

- Pokrenuti OpenROAD gui:

```
sh > openroad -gui
```

- Prvi korak je podešavanje OpenROAD-a. Možete koristiti priloženu skriptu `startup.tcl` da postavite neke opšte pomoćne promenljive i funkcije na koje se druge skripte mogu oslanjati. Pogledajte skriptu da biste videli šta ona radi.

```
OpenROAD > source scripts/startup.tcl
```

  Sada kada je alat podešen, potrebno je da obezbedite informacije o svom projektu. Informacije koje treba da obezbedite su:

[1] Komandna linija omogućava skriptovanje i ponovljivost.

- PDK biblioteke, koje sadrže informacije o tehnološkom procesu i korišćenim bibliotekama.
- Sintetizovana netlista (*synthesized netlist*), koja je generisana na kraju faze sinteze i sadrži informacije o vašem dizajnu.

Podešavanje PDK-a za ovu tehnologiju isto je kao ono koje ste uradili u prošloj vežbi, a za ovu vežbu se automatski izvršava skriptom `init_tech.tcl`, koja se učitava pomoću skripte `startup.tcl` koju ste već pokrenuli.

Sintetizovanu netlistu ste već generisali unutar direktorijuma `yosys/out/`.

- Učitati sintetizovanu netlistu:

```
OpenROAD > read_verilog ../yosys/out/pfe_yosys.v
```

- Unesite komandu:

```
OpenROAD > link_design pfe_chip
```

`link_design` povezuje učitanu Verilog netlisu sa bibliotekama ćelija u OpenROAD okruženju. Ova komanda razrešava svaku instanciranu ćeliju, standardne ćelije i makro blokove, prema Liberty (.lib) definicijama, formira interni graf kašnjenja i proverava ispravnost svih portovskih veza. Mora se pozvati nakon komandi `read_liberty` i `read_verilog`, a pre bilo kakvih komandi za fizički dizajn ili analizu kašnjenja.

- Nakon ovoga, potrebno je da učitamo fajlove potrebne za vremensku analizu:

```
OpenROAD > read_sdc src/constraints.sdc
```

- I možemo da proverimo da da li smo zaboravili da napišemo neka ograničenja i da generišemo izveštaje:

```
OpenROAD > check_setup -verbose > reports/01-01_pfe_checks.rpt

OpenROAD > report_checks -unconstrained -format end \
           -no_line_splits >> reports/01-01_pfe_checks.rpt
OpenROAD > report_checks -path_delay max -format end \
           -no_line_splits >> reports/01-01_pfe_checks.rpt
OpenROAD > report_checks -path_delay min -format end \
           -no_line_splits >> reports/01-01_pfe_checks.rpt
```

- I na kraju čitajte skriptu za povezivanje napajanja:

```
OpenROAD > source scripts/power_connect.tcl
```

Sada smo spremni da krenemo sa fazom *floorplanning*.

Inicijalizacija floorplan-a

Prvi korak je određivanje veličine površine čipa (*die area*) i površine jezgra (*core area*). Otvorite odeljak *Floorplan Initialization* u OpenROAD dokumentaciji: https://openroad.readthedocs.io/en/latest/main/src/ifp/README.html#initialize-floorplan.

- Koliko metoda postoji za zadavanje dimenzija rasporeda (*floorplan*) u OpenROAD-u?
- Navedite primer komande za inicijalizaciju rasporeda za svaku metodu.

- Šta je *core space*?
- Šta je *aspect ratio*?
- Šta je *die area*?
- Šta je *core area*?

Pretpostavite da vam niko nije dao dimenzije čipa. A i nije još uvek :)

- Gde pronalazite informacije o površini jezgra (*core area*)?
- Gde pronalazite informacije o dimenzijama padova?
- Kako određujete površinu čipa (*die area*)? Šta treba uzeti u obzir prilikom njenog određivanja?

Prodiskutovati.

Ključno pitanje prilikom inicijalizacije floorplan-a glasi: kolika je najmanja površina floorplan-a sa kojom je i dalje moguće ispuniti sve zadate specifikacije?

Kao donja granica, neophodna je dovoljno velika površina za smeštanje svih I/O padova, makroa i standardnih ćelija. Sa stanovišta minimizacije površine, idealno bi bilo postaviti standardne ćelije bez ikakvog međusobnog razmaka, tako da u potpunosti popune oblast jezgra (*core area*). Međutim, to je praktično neizvodljivo iz sledećih razloga:

- Broj veza koje mogu proći kroz određenu površinu ograničen je brojem raspoloživih metalnih slojeva, širinom žica i zahtevima za minimalni razmak.
- Vremenske karakteristike u velikoj meri zavise od prostornog rasporeda ćelija. Postavljanje ćelija bez ikakvog razmaka oduzima alatu fleksibilnost pri njihovom raspoređivanju, čime se smanjuju mogućnosti optimizacije, poput grupisanja ćelija sa gustom međusobnom povezanošću.
- Svaki dizajn mora sadržati mrežu za napajanje. Mreža za napajanje ograničava pozicije na kojima ćelije mogu biti smeštene i/ili otežava rutiranje signala, što dodatno povećava potrebnu površinu čipa.
- Većina dizajna zahteva jedan ili više taktnih signala koji se realizuju u obliku stabla taktnog signala. Ovo stablo zahteva dodatnu površinu za bafere neophodne za njegovu realizaciju. Osim toga, algoritam za sintezu stabla takta može postići bolje rezultate ukoliko raspolaže većim prostorom za smeštanje bafera.
- Makro ćelije (engl. *macro cells*), poput SRAM-a u našem primeru, obično zahtevaju dodatni slobodan prostor duž svojih ivica kako bi se pravilno uklopile uz ostale ćelije.

Kao posledica svega navedenog, redovi predviđeni za standardne ćelije ne mogu biti u potpunosti popunjeni. Iskorišćenost (*utilization*) pokazuje koliki procenat redova standardnih ćelija je zauzet. Po pravilu, nije moguće unapred utvrditi da li se svi zahtevi mogu ispuniti pri određenom stepenu iskorišćenosti. I raspoređivanje i rutiranje spadaju u NP-kompletne probleme. Bez prolaska kroz kompletan postupak raspoređivanja i rutiranja, rešenje problema ostaje nepoznato. Stoga je *back-end* proces projektovanja čipova po svojoj prirodi iterativan.

Zadatak 3: Inicijalizacija floorplan-a

- Da bi konačna skripta bila čitljivija i lakša za izmene, često je praktično definisati pomoćne promenljive koje predstavljaju glavne metrike rasporeda (*floorplan*). Na taj način se smanjuju redundantnost i ponavljanje, jer osnovne parametre možete menjati na jednom mestu umesto da budu rasuti po datotekama skripte. Ipak, vodite računa da upotreba prevelikog broja nepotrebnih promenljivih može dovesti do suprotnog efekta i učiniti skriptu teškom za čitanje. Počnimo definisanjem nekoliko promenljivih koje predstavljaju ključne metrike našeg rasporeda. Sve vrednosti su izražene u mikrometrima.

- Definisati dimenzije čipa pomoću:

```
OpenROAD > set chipH    1600; # OR die height (top to bottom)
OpenROAD > set chipW    1600; # OR die width (left to right)
```

- Definišite veličinu *pad driver*-a i veličinu *bonding pad*-a:

```
OpenROAD > set padD      180; # pad depth (edge to core)
OpenROAD > set padW       80; # pad width (beachfront)
OpenROAD > set padBond    70; # bonding pad size
```

- Definišite rezervisani prostor za prsten napajanja:

```
OpenROAD > set powerRing  80; # reserved space for power ring
```

- Zatim izvedite marginu jezgra iz prethodnih promenljivih:

```
OpenROAD > set coreMargin [expr {$padD + $padBond + $powerRing}];
```

- Na kraju, inicijalizujte floorplan sledećom komandom:

```
OpenROAD > initialize_floorplan -die_area "0 0 $chipW $chipH" \
           -core_area "$coreMargin $coreMargin \
           [expr $chipW-$coreMargin] [expr $chipH-$coreMargin]" \
           -site "CoreSite"
```

  Nakon inicijalizacije floorplan-a, trebalo bi da vidite poruku sličnu poruci ispod. Imajte u vidu da stvarna površina jezgra nije tačno ona koja je navedena komandom `initialize_floorplan`, zato što mora biti poravnata sa mrežom *core site*.

```
Core area lower left (330.000, 330.000) snapped to (330.240, 332.640)
```

U ovom trenutku možete da prikažete oblik svog čipa u OpenROAD GUI-ju, što znači da ste uspešno inicijalizovali floorplan. Međutim, u narednim koracima možete shvatiti da ova ograničenja nisu odgovarajuća za vaš čip. Umesto da svaki put ponovo pišete ove komande, možete ih sačuvati u TCL skripti. Što ste već svakako odradili :)

I/O raspoređivanje

Čipovi zahtevaju specijalizovana kola za zaštitu od elektrostatičkog pražnjenja (*ESD*) i snažne bafere za pogon velikih opterećenja van čipa. I/O drajveri uključuju sva ova kola, a takođe sadrže i *bonding pad*-ove koji omogućavaju fizičko povezivanje između čipa i pakovanja. I/O drajveri su u osnovi specijalizovani baferi, mnogo veći od regularnih standardnih ćelija, i projektovani su tako da se postavljaju oko jezgra.

Pored jezgra dizajna i I/O drajvera, *pad frame* će sadržati i padove za napajanje i masu,

kako za jezgro tako i za I/O drajvere, kao i *filler* ćelije i ugaone ćelije za povezivanje ivica *pad frame*-a. Neophodne informacije o I/O ćelijama (fizička veličina, imena pinova, pogonska snaga i slično) nalaze se u PDK-u.

U ovoj vežbi, top modul `pfe_chip` je već napisan za vas.

Zadatak 4:

- Otvorite glavni modul `pfe_chip.sv` u direktorijumu `rtl`.
  - Koji moduli instanciraju padove za napajanje?
  - Koji moduli instanciraju padove ulaznih signala?
  - Koliko je padova za napajanje definisano?
  - Koliko je ukupno padova instancirano?
- Pređite u PDK direktorijum.
  - U kojim datotekama pronalazite geometrijske informacije o padovima?
  - Navedite veličinu, simetriju i *site* padova ulaznih signala.

Zašto je bitan raspored pinova?

Zadatak 5:

- Kao projektant integrisanih kola, morate da odlučite koje ivice ćete koristiti, kako ćete raspodeliti padove po svakoj ivici i kako ćete definisati raspored pinova.
  - Da li je pogodno postaviti padove na samo jednu ivicu? Ako jeste, kada?
  - Koja je glavna prednost korišćenja svih ivica?
  - Kako biste rasporedili padove za napajanje i zašto? Koliko ivica biste koristili za vaš čip?
  - Da li biste signale visoke frekvencije (takt) postavili blizu osetljivih signala (napajanje ili analogni signali)? Zašto?
  - Kako biste rasporedili signale povezane sa istim podmodulima? Blizu jedne drugih ili ne? Zašto?
  - Otvorite `pfe_chip.sv` da biste pročitali nazive padova koje treba rasporediti u svom sistemu.
  - Skicirajte crtež svog čipa na kojem ćete rasporediti padove. Ovo će vam pomoći u sledećem odeljku.
- Prodiskutovati.

Zadatak 6: Postavljanje I/O pinova

- Potrebno je izmeniti TCL skriptu kako biste izvršili komande za raspoređivanje pinova. Šablon se nalazi u `src/pfe_padring.tcl`.

```
# Edge: LEFT (top to bottom)
place_pad -row IO_WEST -location \
           [expr {$westStart -  <i>*$westPitch}] "<ime_pad-a>"

# Edge: BOTTOM (left to right)
place_pad -row IO_SOUTH -location \
           [expr {$southStart +  <i>*$southPitch}] "<ime_pad-a>"
```

```
# Edge: RIGHT (bottom to top)
place_pad -row IO_EAST -location \
           [expr {$eastStart +  <i>*$eastPitch}] "<ime_pad-a>"

# Edge: TOP (right to left)
place_pad -row IO_NORTH -location \
           [expr {$northStart -  <i>*$northPitch}] "<ime_pad-a>"
```

- `<i>` treba zameniti brojem od 0 do 9, dok `<ime_pad-a>` treba zameniti imenima pad-ova iz fajla `pre_chip.sv`. Potrebno je povezati sve pad-ove.
- U ovom trenutku trebalo bi da ste sačuvali sve ove komande u skriptu `src/pfe_padring.tcl`.

  Zatim možete pokrenuti sledeću komandu da učitate `tcl` datoteku.

  ```
  OpenROAD > source src/pfe_padring.tcl
  ```

- Postavimo i redove za napajanje:

  ```
  OpenROAD > make_tracks
  ```

- Definisati visinu standardnih ćelija:

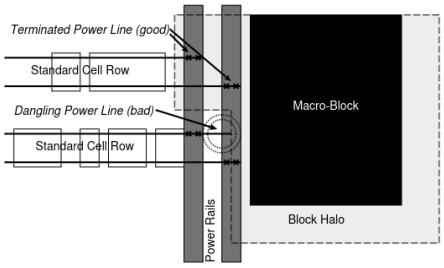
  ```
  OpenROAD > set siteHeight \
              [ord::dbu_to_microns [[dpl::get_row_site] getHeight]]
  ```

Projektovanje "back-end" dela je iterativan proces, tako da kasnije u toku rada možete shvatiti da vaše odluke nisu bile najbolje i ako niste zadovoljni rasporedom pinova, izmenite datoteku `src/pfe_padring.tcl` i ponovo je pokrenite u GUI-ju.

Raspoređivanje makroa

Sledeći korak je raspoređivanje makroa. Prilikom postavljanja makro ćelija, morate odlučiti gde ćete ih smestiti i kako će one biti orijentisane. Da biste doneli takve odluke, morate imati na umu da će makro u potpunosti blokirati neke metalne slojeve i da to lako može stvoriti zagušene oblasti unutar čipa (engl. *congested area*).

Takodje, makro ćeliji je potreban *halo* koji sprečava da standardne ćelije i prsten za napajanje budu postavljeni previše blizu makro ćeliji. Ovo utiče na površinu dostupnu za rutiranje. Videti sliku 3.



Slika 3: *Halo* koji je u konfliktu sa vodovima za napajanje.

Mi koristimo SRAM makro ćelije. SRAM makro će u potpunosti blokirati donja četiri metalna sloja (M1–M4). Preostali metalni slojevi biće dostupni za rutiranje iznad SRAM ćelije, ali TM1

i Metal4 zapravo nisu predviđeni za veliko rutiranje signala. Usled toga, pri postavljanju makroa blizu jedan drugom lako mogu nastati zagušene oblasti. SRAM makroi takođe ne mogu da se rotiraju.

Zadatak 7: Postavljanje SRAM makro ćelija

- Naš dizajn ima dve SRAM makro ćelije. Svaki FIFO bafer ima po jednu.
- Umesto korišćenja ugrađene OpenROAD komande `place_macro`, koristićete pomoćnu funkciju iz `openroad/scripts/floorplan_util.tcl` pod nazivom `placeInstance`, koja podržava dodatne orijentacije.

  Sintaksa komande je `placeInstance name X Y orientation`.

  - Pomoćna funkcija je već učitana u `startup` skripti.

- Podesiti putanje SRAM ćelija

```
OpenROAD > set bank0_sram0 i_fifo_soc/i_fifo_in.gen_sram.i_sram
OpenROAD > set bank1_sram0 i_fifo_soc/i_fifo_out.gen_sram.i_sram
```

- Podesiti veličine SRAM ćelija

```
OpenROAD > set RamMaster256x8 \
           [[ord::get_db] findMaster "RM_IHPSG13_2P_256x8_c2_bm_bist"]
OpenROAD > set RamSize256x8_W [ord::dbu_to_microns [$RamMaster256x8 getWidth]]
OpenROAD > set RamSize256x8_H [ord::dbu_to_microns [$RamMaster256x8 getHeight]]
```

- Nadjimo koordinate jezgra

```
OpenROAD > set coreArea      [ord::get_core_area]
OpenROAD > set core_leftX    [lindex $coreArea 0]
OpenROAD > set core_bottomY  [lindex $coreArea 1]
OpenROAD > set core_rightX   [lindex $coreArea 2]
OpenROAD > set core_topY     [lindex $coreArea 3]
```

- Pomoćne promenljive za postavljanje makro-a

```
OpenROAD > set floorPaddingX  10.0
OpenROAD > set floorPaddingY  10.0
OpenROAD > set floor_leftX    [expr $core_leftX + $floorPaddingX]
OpenROAD > set floor_bottomY  [expr $core_bottomY + $floorPaddingY]
OpenROAD > set floor_rightX   [expr $core_rightX - $floorPaddingX]
OpenROAD > set floor_topY     [expr $core_topY - $floorPaddingY]
OpenROAD > set floor_midpointX \
               [expr $floor_leftX + ($floor_rightX - $floor_leftX)/2]
OpenROAD > set floor_midpointY \
               [expr $floor_bottomY + ($floor_topY - $floor_bottomY)/2]
```

- Razmak izmedju SRAM makro-a po X osi

```
OpenROAD > set sram_gap_x 220
```

- Centriranje makro-a po Y osi:

```
OpenROAD > set Y [expr {$floor_midpointY - $RamSize256x8_H/2}]
```

- Postavljanje leve SRAM ćelije

```
OpenROAD > set X0 [expr {$floor_midpointX - $sram_gap_x/2 - $RamSize256x8_W}]
OpenROAD > placeInstance $bank0_sram0 $X0 $Y R0
```

- Postavljanje desne SRAM ćelije

```
OpenROAD > set X1 [expr {$floor_midpointX + $sram_gap_x/2}]
OpenROAD > placeInstance $bank1_sram0 $X1 $Y R0
```

- Kada su makroi postavljeni, potrebno je da zadate *halo* sledećom komandom.

```
OpenROAD > cut_rows -halo_width_x 2 -halo_width_y 2
```

Treba napomenuti da ste dosta promenljivih mogli da izbacite iz skripte. Recimo mogli smo sami da odredimo koordinate `Y, X0, X1` i da izbegnemo gomilu proračuna.

Planiranje napajanja

Pri projektovanju rasporeda napajanja, cilj je da se napajanje ravnomerno raspodeli do svakog dela dizajna. Pri rutiranju naponskih signala, potrebno je voditi računa o elektromigraciji i IR padovima. Elektromigracija je pojava nastala usled pomeranja atoma, koja može izazvati kratke spojeve, otvorena kola, degradaciju performansi ili otkaz uređaja. Ona je proporcionalna temperaturi i gustini struje koja teče kroz žice, tako da šire žice više ublažavaju pojavu elektromigracije. IR padovi su, sa druge strane, pad napona usled otpornosti žica ($U = I * R$).

Signali `VDD` i `VSS` su globalne veze i uglavnom su povezani sa svim ćelijama. Iako su gornji metalni slojevi pogodniji za distribuciju napajanja kroz čip, standardne ćelije zahtevaju naponske signale na sloju M1. Zbog toga distribucija napajanja prati sledeću hijerarhiju:

- prstenovi (*rings*) provode `VDD` i `VSS` oko čipa (možete ih zamisliti kao obilaznice),
- trake (*stripes*) provode `VDD` i `VSS` preko čipa (možete ih zamisliti kao glavne avenije),
- vodovi (*rails*) dovode `VDD` i `VSS` do standardnih ćelija (možete ih zamisliti kao ulice).

Pri projektovanju rasporeda napajanja, morate odlučiti na kojim slojevima ćete postaviti prstenove, trake i vodove. Za svaki sloj, takođe treba odrediti korak (*pitch*) i razmak (*spacing*). Korak je rastojanje između dva susedna `VDD` ili dva susedna `VSS` voda u metalnom sloju. Razmak je minimalno rastojanje između susednih `VDD` i `VSS` vodova. Vertikalne veze između metalnih slojeva nazivaju se *vias*. Razmak i korak utiču na integritet napajanja i na zagušenje pri rutiranju. Pored toga, neki elektromagnetni efekti (*crosstalk*, kapacitivna sprega) mogu se pojaviti kod loše raspoređenog napajanja, što degradira ukupne performanse.

Zadatak 8: Napajanje

- Radi jednostavnosti vežve, skripta za povezivanje napajanja vam je već data i možete je pozvati:

```
OpenROAD > source scripts/power_connect.tcl
```

- Šta se promenilo u vasem floorplan-u?
- Pogledati skriptu `power_connect.tcl`. Koje su vrednosti koraka (`pitch`) i razmaka (*spacing*)?

- U kojim metalima je prsten napajanja (*power ring*)? U kojem sloju su trake (*stripe*) a u kojem sloju metala su vodovi (*rails*)?

Zadatak 9: Checkpoint

- Sačuvati trenutno stanje dizajna:

```
OpenROAD > save_checkpoint 01_pfe.floorplan
OpenROAD > report_image "01_pfe.floorplan" true
```

Floorplan done!